

Problem:

Two mismatched groups exist inside ALU at the same time: students who want internship-style experience but mostly see openings at large, external organizations, and student-led ventures who need hands-on help (dev, design, marketing, ops, research) but have no structured, trustworthy channel to reach ALU talent specifically. Generic job boards don't fit either side well, they're not scoped to ALU.

VentureConnect is deliberately a closed, ALU-scoped marketplace, not a general job board:

- Startups must be admin-verified before they can post anything
- Students get one place to discover, apply to, and track the status of opportunities across every ALU venture, instead of chasing individual startups.
- Startups get a lightweight applicant pipeline (Applied → Under review → Interview → Accepted/Rejected) sized for a handful of applicants per posting, not an enterprise tracking system.

System Design:

The app follows a layered, feature-first architecture: a screens layer (pure UI), a Bloc/Cubit layer (state management, one per feature), and a repository layer (all Firestore/Firebase Auth calls), talking to Firebase as the sole backend.

Only two Firebase products are used: Firebase Authentication (email/password and Google Sign-In) and Cloud Firestore (all application data, real-time) with Bloc and Cubit used for state management. **Bloc** used for reactive state control where many things could happen based on the final state received and **Cubit** for simple state management where there is just one truth e.g clicking a button always leads to another page just data changes.

Every feature follows the same shape regardless of which is used: a Repository exposes Firestore streams and CRUD methods and is the only thing that touches Firebase directly; the Bloc/Cubit just get what it needs from the repo and translates that into UI-facing state.

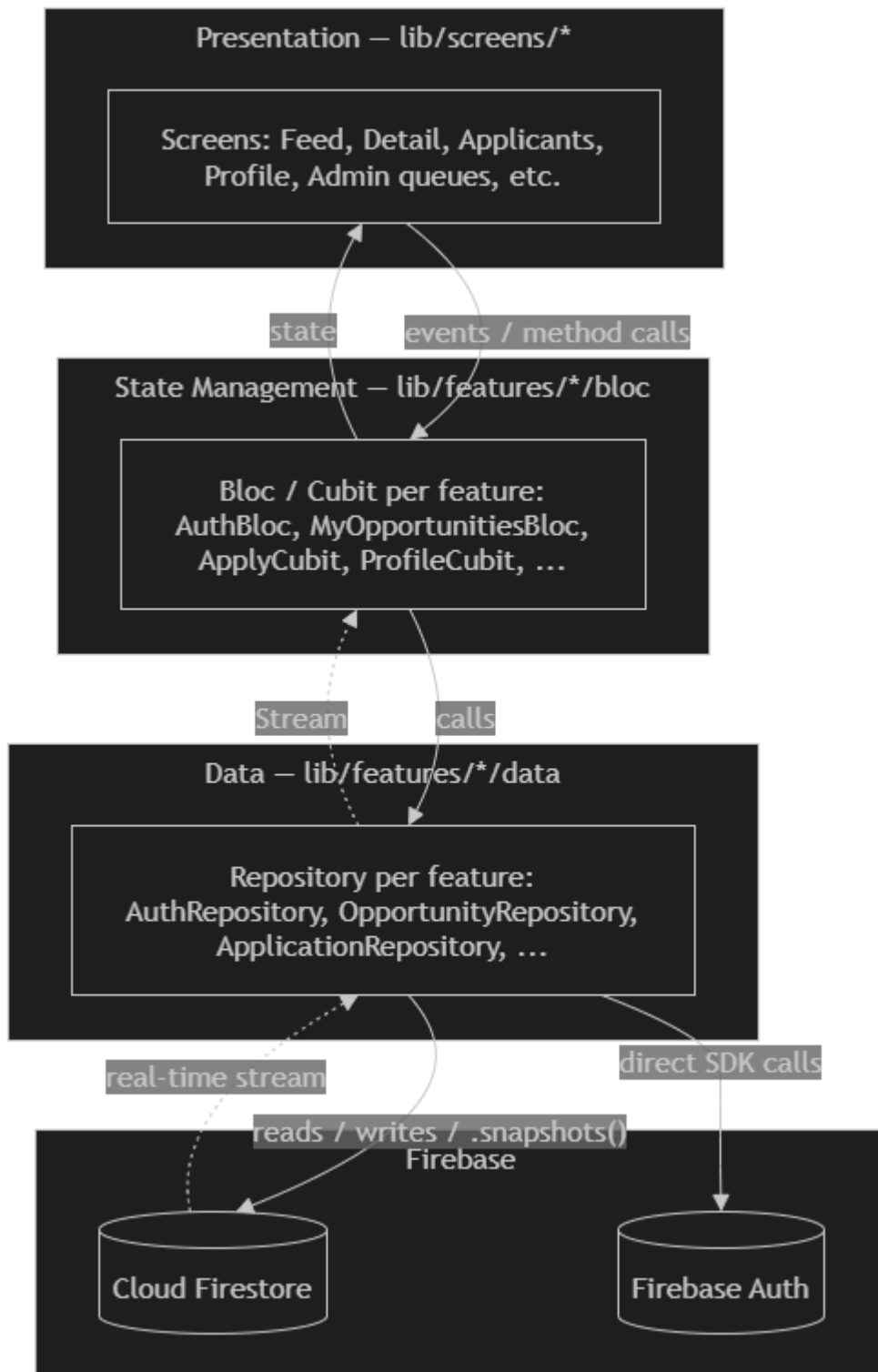
UI/UX Reasoning:

- **Per-role bottom navigation:** Each role's home screen shows genuinely different information architecture (student: Discover/Applications/Saved/Profile; startup: Postings/Profile; admin: Verifications/Users). Forcing one shared nav shape and hiding/disabling tabs per role would have been worse UX than just giving each role the nav it actually needs.

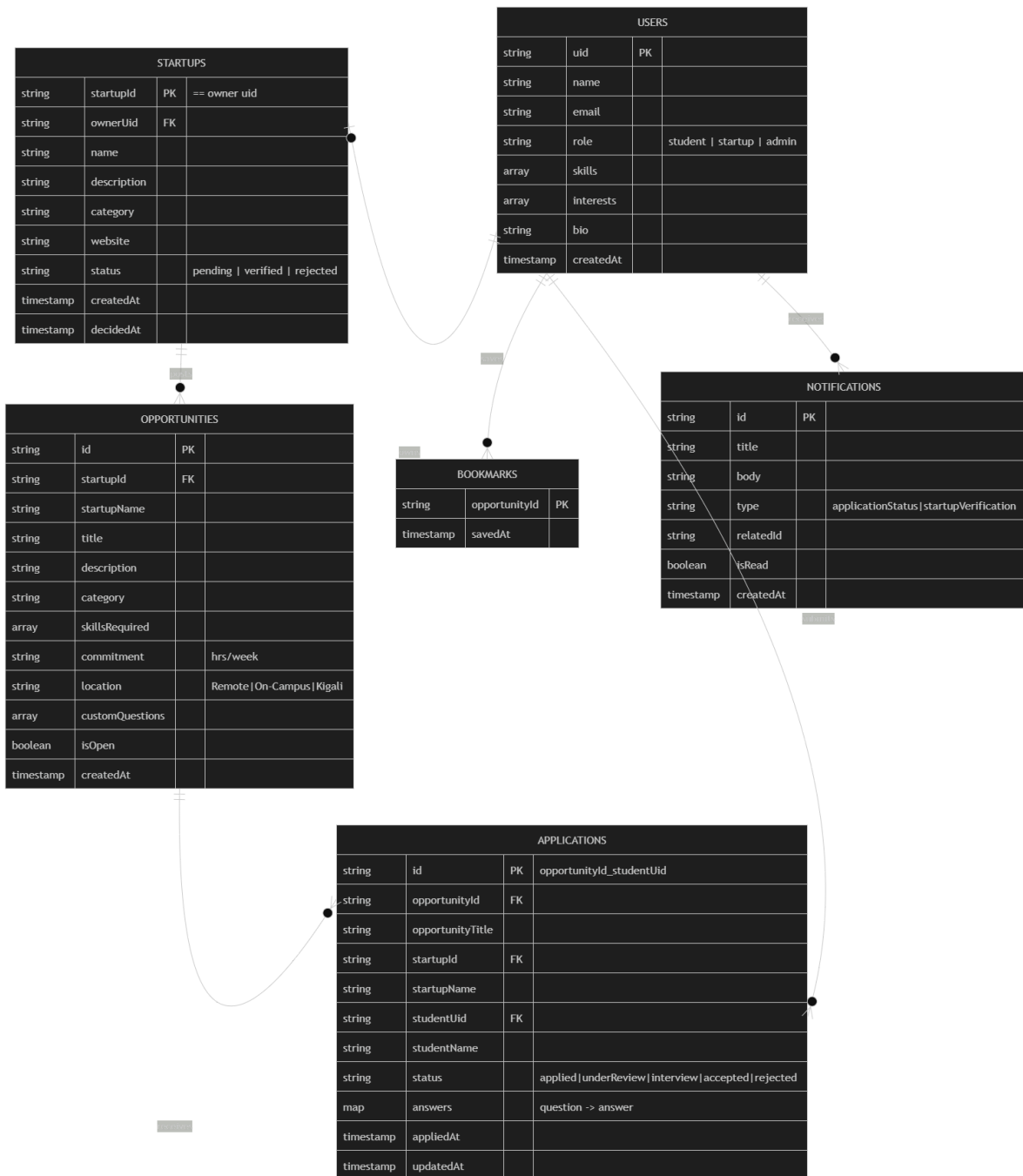
- **Loading (empty, error states) are first-class, everywhere:** Every list-driven screen has a dedicated LoadingView, EmptyState, and ErrorView rather than a blank screen or a raw exception.

Technical Decisions:

- **Use of FirebaseAuth/ Google Provider:** It streamlines authentication not needing to write code for authentication as it is auto handled by Firebase. The google provider also makes it easier to interact with the app as most users in today's world have a google account.
- **Bloc/Cubit State Managers:** I used these for my state management. Bloc does the heavyweight state management, wherever a feature reacts to more than one kind of trigger over its lifetime e.g After a user logs in Bloc state manager decides which homescreen is shown based on the role of the user. And Cubit was used for lightweight state management, for narrow, single-purpose state e.g a student applies for an opportunity the status changes to applied.



Database Schema



Scalability Considerations:

- **Real-time by default:** Every list-driven screen is backed by a Firestore `.snapshots()` stream, not a one-time fetch. There is no pull-to-refresh

anywhere in the app because there's nothing to refresh. This holds up the same way whether five people or five hundred are using the app concurrently.

- **Denormalized reads:** Opportunity stores startupName; Application stores both opportunityTitle and startupName. This trades a small amount of write-time duplication for read-time simplicity: list and detail screens never join across collections client-side, so query cost doesn't grow with the number of relationships in the data.
- **Client-side joins where Firestore has no server-side join:** The Saved-opportunities screen and the admin User Management screen both combine two independent streams (bookmarked IDs + all opportunities; all users + all startups) by keeping the latest value of each and recomputing a merged list whenever either updates. This is the correct pattern given Firestore's lack of joins without the enterprise version, and stays cheap at this app's scale since both source collections are small.
- **Feature-first modular structure:** lib/features/<feature>/ {data,models,bloc} holds logic only; lib/screens/<feature>/ holds UI only. Adding a new feature later (e.g. messaging) means adding one new folder pair without touching existing ones.

Challenges:

- **First Google sign-in had no Firestore profile:** Added a distinct AuthNeedsRoleSelection state to AuthBloc and a one-time role-selection screen, rather than treating "no profile yet" as an error.
- **Changed the project name during building:** This caused errors when importing files so switched to path imports over package imports.

Testing:

Use of flutter analyze and flutter test to make sure the app was running with no error before committing each feature to github and the entire app tested end to end per feature release.

Limitations:

- **No Firebase Storage:** No support for file / image uploads to storage due to it being a paid feature. So profile photos are generated initials only.
- **No push notifications:** The notification center is in-app/Firestore-backed only; nothing reaches a user whose app isn't open or recently connected.
- **Client-side sorting assumes moderate data volume:** It's the right trade-off at this app's realistic scale. but a collection growing into the tens of thousands of documents would need either composite indexes or server-side pagination.

- **Admin role assignment is a manual:** Not an in-app workflow, intentional as seeding it will make the user auto log in on start of the app which isn't intended.

Demo Video Link: [Demo_video](#)

Github Link: [Venture_Connect](#)